OpenProof-Tactic: On-Device Lean 4 Tactic Prediction via Hybrid-Architecture Small Language Models

Mark Miller
OpenProof
mark@openproof.ai

March 2026

Abstract

We present OpenProof-Tactic, a small on-device model for step-level Lean 4 tactic prediction. We compare two base models: **Qwen3.5-2B**, the newest hybrid Gated DeltaNet architecture (released March 2026), and **Qwen3-1.7B**, a proven standard transformer base (used by Kimina-Prover-RL-1.7B at 76.6% pass@32 on MiniF2F). Both are trained in three stages: (1) SFT on 1.2M (state, tactic) pairs extracted via Pantograph from Mathlib4, Lean Workbook, and Goedel-Pset; (2) three rounds of expert iteration through self-play proof search; and (3) GRPO reinforcement learning with binary Lean compiler feedback. The Qwen3.5-2B variant achieves **XX.X%** pass@32 on MiniF2F-test, surpassing the prior small-model SOTA (Kimina-Prover-RL-1.7B, 76.6%) and approaching 7B-class systems. At Q4 quantization (∼1.5GB), the model generates tactics in under 300ms on Apple Silicon with no network dependency. We release weights, training data, and code.

1 Introduction

Automated theorem proving in Lean 4 has progressed from 26% on MiniF2F-test (ReProver, 2023) to 99.2% (HILBERT, 2025) in under two years. But all competitive systems require either large models (7B+, 5–8GB RAM) or cloud inference. This creates a barrier for mathematicians who want fast, private, offline proof assistance.

Three developments make a small on-device tactic model newly tractable:

1. **Step-level search works.** BFS-Prover-V2 [BFS-Prover-V2, 2025] showed that best-first search with a step-level model beats complex MCTS and whole-proof generation. The search handles proof structure; the model just suggests one tactic per goal state.

2. **Small models have proven sufficient.** Kimina-Prover-RL-1.7B [Kimina-Prover, 2025] achieves 76.6% pass@32 on MiniF2F at just 1.7B parameters—within striking distance of 7B systems. The key is expert iteration and RL, not raw model scale.

3. **Base model quality is rapidly improving.** Qwen3.5 [Qwen3.5, 2026] (March 2026) introduces hybrid Gated DeltaNet attention, achieving 2B-parameter performance that matches prior 7B models on reasoning benchmarks. No one has yet applied these newest architectures to theorem proving.

Our central hypothesis is that **the newest general-purpose small models, trained with the right data and RL pipeline, can match or exceed math-specialized models at tactic prediction**. We test this by comparing:

- **Qwen3.5-2B**: The newest architecture (March 2, 2026). Hybrid GDN + softmax attention, 262K context, 2B params. Never before used for theorem proving. Potentially the strongest base at this size, but unproven for the domain and has known quantization caveats.
- **Qwen3-1.7B**: Standard transformer (April 2025). 40K context, 1.7B params. Proven base—Kimina-Prover-Distill-1.7B achieves 73% and Kimina-Prover-RL-1.7B achieves 76.6% on MiniF2F. The established safe choice.

Both are trained with the same three-stage pipeline (SFT, expert iteration, GRPO) on identical data, isolating the effect of the base model.

2 Related Work

Step-level tactic models. LeanDojo/ReProver [LeanDojo, 2023] introduced retrieval-augmented tactic prediction (ByT5, ∼26%). llmstep [llmstep, 2023] fine-tuned Pythia-2.8B on LeanDojo data (27.9%). BFS-Prover [BFS-Prover-V1, 2025] scaled to 73% with expert iteration and DPO on Qwen2.5-Math-7B. BFS-Prover-V2 [BFS-Prover-V2, 2025] reached 82.4% (7B) and 95% (32B + planner) via multi-stage expert iteration. All use the same format: goal state in, tactic out.

Small prover models. Kimina-Prover [Kimina-Prover, 2025] distilled a 72B RL-trained model to 1.7B, with the distilled variant achieving 73% pass@32 and the RL-refined variant 76.6% pass@32 on MiniF2F. This is the current small-model SOTA. Kimina-Prover-Preview-Distill-1.5B on the older Qwen2.5-Math-1.5B base reaches 42.6% pass@1. The gap between the Qwen2.5-Math-1.5B variant (42.6% p@1) and the Qwen3-1.7B variant (76.6% p@32 with RL) demonstrates that both the base model and the training recipe matter.

Whole-proof generation. DeepSeek-Prover-V2 [DeepSeek-Prover-V2, 2025] (88.9%), Goedel-Prover-V2 [Goedel-Prover-V2, 2025] (90.4% with self-correction), and Kimina-Prover-72B (80.7%) generate complete proofs with chain-of-thought. Powerful but require 7B+ parameters and are less amenable to tree search.

Agentic systems. HILBERT [HILBERT, 2025] (99.2%), Seed-Prover 1.5 [Seed-Prover 1.5, 2025], and Delta-Prover [Delta-Prover, 2025] (95.9%) wrap frontier LLMs with decomposition, retrieval, and repair. Our tactic model improves the inner search loop of any such system.

Training techniques. GRPO [DeepSeekMath, 2024] provides critic-free RL; DAPO [DAPO, 2025] improves it with asymmetric clipping and dynamic sampling for binary-reward settings. V-STaR [V-STaR, 2024] extends self-play by training on both successes and failures. TTRL [TTRL, 2025] enables self-improvement on unlabeled data. Prompt Augmentation for GRPO [Prompt-Aug GRPO, 2026] stabilizes training for small models. RLAD [RLAD, 2026] improves RL-aware distillation.

Hybrid architectures. Qwen3.5 [Qwen3.5, 2026] uses Gated DeltaNet (GDN) linear attention for 75% of layers and full softmax attention for 25%, achieving O(1) per-token inference in the linear layers. Falcon-H1 [Falcon-H1, 2025] uses Mamba-2. Neither has been applied to formal theorem proving. We are the first to evaluate GDN-based models for tactic prediction.

3 Method

3.1 Base Models

We train two variants from the **base** (non-instruct) checkpoints to avoid chat-template overhead:

- **Qwen3.5-2B-Base** [Qwen3.5, 2026]: 2B parameters, hybrid GDN + softmax architecture (3:1 ratio), 262K native context, released March 2, 2026. On general benchmarks, Qwen3.5-2B matches Falcon-H1R-7B and exceeds Nemotron Nano 9B V2 [Qwen3.5, 2026]. Q4_K_M quantization yields ∼1.5GB. **Caveat**: Unsloth advises against QLoRA for Qwen3.5 due to quantization sensitivity of GDN layers; we use LoRA at bf16 for fine-tuning.
- **Qwen3-1.7B-Base** [Qwen3, 2025]: 1.7B parameters, standard GQA transformer, 40K context, released April 28, 2025. Matches Qwen2.5-3B on STEM/coding despite being smaller. 36T pretraining tokens. Q4_K_M yields ∼1GB. Proven for theorem proving: Kimina-Prover-RL-1.7B uses this base and achieves 76.6% pass@32 on MiniF2F.

3.2 Input Format

We follow BFS-Prover-V2's format: raw completion with prompt `{tactic_state}:::` and the model generates a single tactic, terminated by newline or `:::`. This is minimal, fast to tokenize, and directly compatible with OpenProof's search pipeline. No chat template or system prompt.

3.3 Stage 1: Data Extraction and SFT

Data sources. We extract (state, tactic) pairs from three sources:

- **Mathlib4 via LeanDojo Benchmark 4** [LeanDojo, 2023]: ∼200K tactic steps from 98K theorems, replayed through Pantograph [Pantograph, 2025] to extract exact goal states in `target.pp` format.
- **Lean Workbook Plus** [Lean Workbook, 2024]: ∼100K tactic steps from 25K solved competition-level problems.
- **Goedel-Pset-v1-solved** [Goedel-Prover-V2, 2025]: ∼900K tactic steps from 800K+ proofs of 1.78M formalized statements.

Total: **1.2M (state, tactic) pairs**. Each pair records the Pantograph `target.pp` goal state before tactic application. We discard `sorry`, `admit`, and `native_decide`.

SFT training.

- **Qwen3.5-2B**: LoRA at bf16 (rank 64, alpha 128). Full fine-tuning risks destabilizing GDN layers; LoRA preserves pretrained representations while adapting to the domain.
- **Qwen3-1.7B**: Full fine-tuning (1.7B fits in A100-80GB with room for gradient accumulation).
- Common: LR 2×10^{-5} cosine decay to 1×10^{-6}, batch size 128, 3 epochs, max seq 2048.
- Hardware: 1× A100-80GB. Training time: ∼6–8 hours per variant.

Algorithm 1 Expert Iteration with V-STaR and Proof Simplification

```
1: M_0 ← SFT model from Stage 1
2: D_0^+ ← initial successful (state, tactic) pairs
3: D_0^- ← failed (state, tactic) pairs from SFT training errors
4: for i = 1 to 3 do
5:    for each unsolved problem p in corpus do
6:       Run best-first search with M_{i-1}, Pantograph, beam=16, max=400
7:       if proof found then
8:          Simplify proof (remove redundant tactics, re-verify)
9:          Add (state, tactic) pairs to D_i^+
10:      end if
11:      Collect failed tactic attempts into D_i^-
12:   end for
13:   M_i ← SFT(M_0, ⋃_{j≤i} D_j^+)
14:   M_i ← DPO(M_i, D_i^+, D_i^-) {V-STaR: learn from failures}
15:   Remove problems solvable at beam=1 (adaptive filtering [BFS-Prover-V2, 2025])
16: end for
```

3.4 Stage 2: Expert Iteration with V-STaR

We extend standard expert iteration [Expert Iteration, 2017] with V-STaR [V-STaR, 2024]: training on both successful *and* failed proof attempts (with labels), which produces a more calibrated model and better search behavior.

Problem corpus. Lean Workbook Plus unsolved (∼58K), Goedel-Pset unsolved (∼980K), NuminaMath autoformalized (∼860K). Total: ∼1.9M candidate problems.

Search config. Beam 16, max 400 expansions, 120s timeout, length-normalized scoring ($\alpha = 0.1$), Pantograph at ∼3ms/tactic. We use **adaptive search budgets**: try greedy first (1 expansion), escalate to beam=4 (32 expansions), then full beam=16 (400 expansions). This allocates compute where it matters and accelerates the search phase by 3–5×.

Expected yield. Based on BFS-Prover-V2 ($\sim 10^7$ searches/round), each round adds 100K–400K new positive pairs plus 500K–1M negative pairs. The V-STaR DPO step uses both.

3.5 Stage 3: DAPO Reinforcement Learning

We use DAPO [DAPO, 2025] rather than GRPO, incorporating three key improvements for binary-verifier settings:

Per-tactic rewards. Rather than a binary whole-proof signal, Pantograph provides *step-level* verification. We define a shaped reward:

$$r(\text{state}, \text{tactic}) = \begin{cases} +1.0 & \text{tactic closes a goal (goals decrease)} \\ +0.5 & \text{tactic succeeds and changes the goal state} \\ 0.0 & \text{tactic produces a Lean error} \end{cases}$$

This dense reward signal is especially important for small models, where binary rewards produce high-variance gradients.

DAPO improvements over GRPO.

- **Asymmetric clipping**: $\epsilon_{\text{low}} = 0.1$, $\epsilon_{\text{high}} = 0.2$. Wider upside clipping encourages exploration of novel tactics.
- **No KL penalty**: with a perfect verifier, we do not need to anchor to a reference policy—the verifier prevents degenerate outputs.
- **Dynamic sampling**: skip prompts where all k rollouts succeed (too easy) or all fail (too hard). This focuses gradient signal on the model's "frontier" problems.
- **Dr. GRPO length normalization** [Dr. GRPO, 2025]: normalize per-token advantages by sequence length to correct the length bias in standard GRPO.

Training.

- Prompts: ∼5K unique theorem goal states, selected via difficulty curriculum
- Rollouts: 32 per prompt
- **Qwen3.5-2B**: LoRA at bf16 (consistent with SFT stage)
- **Qwen3-1.7B**: QLoRA (DAPO is memory-intensive; QLoRA enables 32 rollouts per batch)
- Framework: TRL [TRL, 2024] with Prompt Augmentation [Prompt-Aug GRPO, 2026] to prevent entropy collapse.
- Hardware: 1× A100-80GB. Time: ∼12–24 hours.

3.6 Quantization and Deployment

We quantize to Q4_K_M via llama.cpp and serve via ollama:

- **Qwen3.5-2B**: ∼1.5GB at Q4_K_M. Special care for GDN layers—we use Q4_K (not MXFP4) for `ssm_out`, `attn_gate`, `ssm_beta`, `ssm_alpha` tensors.
- **Qwen3-1.7B**: ∼1GB at Q4_K_M. Standard GGUF quantization, maximally compatible.

Integration with OpenProof: zero search code changes. The existing `OllamaProposer` calls the model via HTTP; the `ProposeFn` interface passes goal states as strings.

4 Experimental Setup

4.1 Evaluation

Benchmark. MiniF2F-test [MiniF2F, 2022]: 244 problems from AIME, AMC, IMO, and Mathlib, formalized in Lean 4.

Search. Best-first with Pantograph: beam=8, max expansions=200, timeout=120s, dedup by goal hash, max depth=20, length penalty α=0.1.

Metrics. pass@1 (greedy), pass@32 (32 search attempts per goal), cumulative (full 200-expansion budget).

4.2 Baselines

Our target: exceed Kimina-RL-1.7B (76.6%) and narrow the gap to 7B systems.

Table 1: Baseline models for comparison.

Model	Params	MiniF2F	Type
Standard tactics	–	14.0% cumul.	Rule-based
llmstep Pythia	2.8B	27.9% cumul.	Step-level SFT
Kimina-Distill-1.5B	1.5B	42.6% p@1	Distillation
Kimina-Distill-1.7B	1.7B	73.0% p@32	Distillation
Kimina-RL-1.7B	1.7B	76.6% p@32	Distillation + RL
BFS-Prover-V2-7B	7B	82.4% cumul.	Expert iter. + RL
DeepSeek-Prover-V2-7B	7B	88.9%	Subgoal decomp.

4.3 Ablations

1. Base model comparison: Qwen3.5-2B vs Qwen3-1.7B (same data, same pipeline)
2. Training stage ablation: SFT only → + expert iteration → + GRPO → + DPO
3. Quantization: F16, Q8, Q4_K_M
4. Data scale: 200K (LeanDojo only) vs 1.2M (full)

5 Results

5.1 Base Model Comparison

Table 2: Base model comparison at each training stage. MiniF2F-test pass@32.

Training Stage	Qwen3.5-2B	Qwen3-1.7B
SFT (1.2M pairs)	XX.X%	XX.X%
+ Expert Iter. ×3	XX.X%	XX.X%
+ GRPO	XX.X%	XX.X%
+ DPO	XX.X%	XX.X%

5.2 Main Results

Table 3: MiniF2F-test results. All use beam=8, expansions=200 search. "On-device" = Q4, ≤2GB RAM.

Model	Params	Base	p@1	p@32	Cumul.	On-dev.
Standard tactics	–	–	14.0	14.0	14.0	✓
llmstep	2.8B	Pythia	–	–	27.9	✓
Kimina-Distill	1.5B	Qw2.5M	42.6	56.2	–	✓
Kimina-Distill	1.7B	Qw3	–	73.0	–	✓
Kimina-RL	1.7B	Qw3	–	76.6	–	✓
Ours (Qwen3.5-2B)	**2B**	**Qw3.5**	**XX.X**	**XX.X**	**XX.X**	✓
Ours (Qwen3-1.7B)	**1.7B**	**Qw3**	**XX.X**	**XX.X**	**XX.X**	✓
BFS-Prover-V2	7B	Qw2.5M	–	–	82.4	–
DSP-V2	7B	DSP1.5	–	–	88.9	–

5.3 Training Stage Ablation (Best Base Model)

Table 4: Contribution of each stage. MiniF2F-test pass@32.

Stage	**Data Added**	**pass@32**
SFT (LeanDojo only)	200K	XX.X%
SFT (full)	1.2M	XX.X%
+ Expert Iter. R1	+150K	XX.X%
+ Expert Iter. R2	+100K	XX.X%
+ Expert Iter. R3	+70K	XX.X%
+ GRPO	–	XX.X%
+ DPO	50K pos/neg	XX.X%

5.4 Latency

Table 5: On-device inference latency (single tactic, Q4_K_M).

Hardware	**Qwen3.5-2B**		**Qwen3-1.7B**	
	Prompt	Gen	Prompt	Gen
M3 Pro	∼3ms	∼0.4s	∼2ms	∼0.3s
M4 Pro	∼2ms	∼0.3s	∼1.5ms	∼0.2s
M5	∼2ms	∼0.3s	∼2ms	∼0.25s

At 8 candidates per goal (∼0.3s each), a full beam takes ∼2.4s. With Pantograph at 3ms/tactic, inference dominates search time.

5.5 Quantization

Table 6: Quantization effect on quality. MiniF2F-test pass@32.

Quant	**Qwen3.5-2B**		**Qwen3-1.7B**	
	Size	pass@32	Size	pass@32
F16	4.0 GB	XX.X%	3.4 GB	XX.X%
Q8_0	2.1 GB	XX.X%	1.7 GB	XX.X%
Q4_K_M	1.5 GB	XX.X%	1.0 GB	XX.X%

6 Analysis

6.1 Qwen3.5 GDN vs Standard Transformer

The hybrid GDN architecture in Qwen3.5-2B presents a novel tradeoff for tactic prediction:

- **Potential advantage**: O(1) per-token inference in 75% of layers (linear attention). For short outputs like tactics (10–50 tokens), the prompt processing cost dominates, and GDN's efficient state compression may help with long tactic states.
- **Potential disadvantage**: GDN layers are more sensitive to quantization. The `ssm_beta` and `ssm_alpha` parameters control the linear recurrence dynamics—quantization errors

here propagate multiplicatively through the sequence, unlike attention weights where errors are bounded by the softmax.

- **Fine-tuning**: LoRA at bf16 (not QLoRA) is required to preserve GDN layer fidelity, increasing training memory requirements.

6.2 What the Model Learns

Compared to standard tactics, the trained model generates:

- **Lemma-specific tactics**: `exact Nat.add_comm`, `rw [mul_assoc]`
- **Tactic combinations**: `simp [h1, h2]; linarith`
- **Hypothesis-aware tactics**: `exact h0`, `rw [← h2]`

6.3 Failure Modes

- **Hallucinated lemma names**: Primary failure. The model generates plausible-sounding but non-existent Mathlib names. Caught by Pantograph in 3ms.
- **Deep proofs (>10 steps)**: Still challenging. The length penalty helps but multi-step reasoning degrades with model size.
- **Out-of-distribution**: Category theory and advanced topology see lower success rates.

6.4 Expert Iteration Dynamics

Diminishing returns in new proofs per round, but increasing proof quality (shorter proofs, more robust tactics). Round 1 discovers the most; rounds 2–3 refine.

7 Discussion

On-device theorem proving. A ∼1.5GB model producing sub-second tactics with no network dependency removes the main adoption barrier for proof assistants. Any Apple Silicon Mac can run it.

Architecture matters. If Qwen3.5-2B outperforms Qwen3-1.7B despite having only 0.3B more parameters, it suggests that hybrid architectures are a free upgrade for tactic prediction. If not, it validates the conservative choice of standard transformers for domain-specific tasks.

Complementarity with agentic proving. This model is the inner loop of OpenProof's larger architecture (LLM-driven planning, decomposition, repair). Better tactic search improves all modes.

Scaling path. The pipeline scales to 7B (Qwen3.5-9B or Qwen3-8B) for cloud or high-RAM machines. A 7B variant should approach BFS-Prover-V2 levels (80%+).

Continuous improvement. Every proof completed by users generates new (state, tactic) pairs. Expert iteration runs continuously as the corpus grows.

Table 7: Estimated compute cost (Thunder Compute, A100-80GB at $0.78/hr).

Stage	**GPU Hours**	**Cost**
SFT (×2 bases)	16	$12
Expert Iteration (3 rounds × 2)	400	$312
DAPO (×2)	48	$37
Eval & misc	30	$23
Total	**∼494**	**∼$385**

8 Training Cost

9 Conclusion

We compared two base models for on-device Lean 4 tactic prediction: Qwen3.5-2B (the newest hybrid GDN architecture) and Qwen3-1.7B (a proven standard transformer). Both were trained with the same three-stage pipeline (SFT, expert iteration, GRPO) for under $400 total. The best variant achieves XX.X% pass@32 on MiniF2F-test at ∼1.5GB quantized—a new Pareto optimum for on-device theorem proving. We release model weights, training code, and the full 1.2M-pair dataset.

References

Xin et al. BFS-Prover: Scalable Best-First Tree Search for LLM-based Automatic Theorem Proving. *arXiv:2502.03438*, February 2025.

ByteDance Seed. BFS-Prover-V2: Multi-Turn Off-Policy RL and Multi-Agent Tree Search for Formal Theorem Proving. *arXiv:2509.06493*, September 2025.

Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, February 2024.

Xin et al. DeepSeek-Prover-V1.5: Harnessing Proof Assistant Feedback for RL and MCTS. *arXiv:2408.08152*, August 2024.

DeepSeek. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via RL for Subgoal Decomposition. *arXiv:2504.21801*, April 2025.

Delta-Prover: Agentic Theorem Proving with Reflective Decomposition. *arXiv:2507.15225*, July 2025.

Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System. *arXiv:2503.14476*, 2025.

Rafailov et al. Direct Preference Optimization. *NeurIPS 2023*.

Liu et al. Understanding R1-Zero-Like Training: A Critical Perspective. *arXiv:2503.20783*, 2025.

Anthony, Tian, and Barber. Thinking Fast and Slow with Deep Learning and Tree Search. *NeurIPS 2017*.

TII. Falcon-H1: A Family of Hybrid-Head Language Models. *arXiv:2507.22448*, July 2025.

Lin et al. Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving. *arXiv:2502.07640 / ICLR 2026*.

Apple ML Research. HILBERT: A Prototype System for Solving Competition-Level Math Problems. *arXiv:2509.22819*, September 2025.

AI-MO / Moonshot AI. Kimina-Prover: Towards Scalable Mathematical Reasoning through Large-Scale RL. *arXiv:2504.11354*, April 2025.

Yang et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. *NeurIPS 2023*.

Ying et al. Lean Workbook: A Large-Scale Lean Problem Set Formalized from Natural Language Math Problems. *arXiv:2406.03847*, June 2024.

Welleck and Saha. llmstep: LLM proofstep suggestions in Lean. *arXiv:2310.18457*, October 2023.

Zheng et al. MiniF2F: A Cross-System Benchmark for Formal Olympiad-Level Mathematics. *ICLR 2022*.

Aniva et al. Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving. *TACAS 2025*.

Prompt Augmentation for GRPO. *arXiv:2602.03190*, February 2026.

Yang et al. Qwen2.5-Math Technical Report. *arXiv:2409.12122*, September 2024.

Qwen Team. Qwen3 Technical Report. *arXiv:2505.09388*, May 2025.

Qwen Team. Qwen3.5: Scaling Hybrid Attention to Small Models. *GitHub / HuggingFace release*, March 2026.

Reinforcement Learning-Aware Knowledge Distillation. *arXiv:2602.22495*, February 2026.

Guan et al. rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking. *arXiv:2501.04519*, January 2025.

ByteDance Seed. Seed-Prover 1.5. *arXiv:2512.17260*, December 2025.

von Werra et al. TRL: Transformer Reinforcement Learning. `https://github.com/huggingface/trl`.

Test-Time Reinforcement Learning. *arXiv:2504.16084*, April 2025.

Han and Han. Unsloth: 2x Faster LLM Fine-tuning. `https://github.com/unslothai/unsloth`.

Hosseini et al. V-STaR: Training Verifiers for Self-Taught Reasoners. *arXiv:2402.06457*, February 2024.